Product of Functions in a Compressed Format

Exploring Elementwise Multiplication of Quantics Tensor-Train Networks

Zhaonan Meng, Miles Stoudenmire

Center for Computational Quantum Physics, Flatiron Institute

FLATIRON INSTITUTE

Introduction

- Continuous multivariate functions arise in all fields of science: $\mathbf{u} = (u_1, \dots, u_n) \longmapsto f(\mathbf{u})$ (u_i denoting n variables).
- By encoding variable u_i through R binary digits, the $f(\mathbf{u})$ can be expressed as a high-dimensional tensor.
- The tensor is compressible via the tensor-train (TT) network, referred to as the Quantics Tensor Train (QTT).
- QTT has been used in quantum physics and fluid dynamics to overcome computational bottlenecks of solving partial differential equations (PDEs) [1].

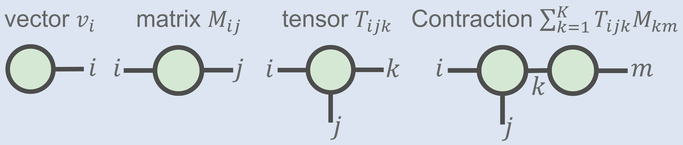
Our Research Question:

Given two functions $f_1(x)$ and $f_2(x)$ represented in QTT form, how to utilize QTTs to compute:

$$g(x) = f_1(x) \cdot f_2(x).$$

Although nonlinear operations like function multiplication are common in PDEs, their implementation in the QTT format is nontrivial and restricts its practical use. **Our research: developing an efficient algorithm to multiply two QTTs.**

Quantics Tensor Train (QTT)



- Discretize a function $f(x)$ ($x \in [0,1)$) in a grid of 2^R equally spaced points. The x is represented in binary $(0.\sigma_1\sigma_2 \dots \sigma_R)_2$:

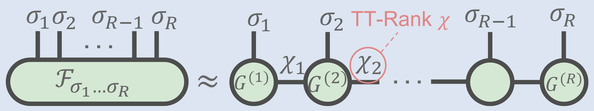
$$0.\sigma_1\sigma_2 \dots \sigma_R = \frac{\sigma_1}{2} + \frac{\sigma_2}{2^2} + \dots + \frac{\sigma_R}{2^R}, \sigma_r \in \{0,1\} \text{ for } r = 1, \dots, R$$

The discretized $f(x)$ is essentially an Rth-mode tensor:

$$f(x) = f(\sigma_1, \dots, \sigma_R) = F_{\sigma_1, \dots, \sigma_R}.$$

- The tensor is approximated via *tensor train (TT)*: decomposed into a sequence of lower-dimensional tensors (*TT-cores*):

$$F(\sigma_1, \dots, \sigma_R) \approx \sum_{\alpha_0, \dots, \alpha_R}^{\chi_1, \dots, \chi_R} G^{(1)}(\alpha_0, \sigma_1, \alpha_1) \dots G^{(R)}(\alpha_{R-1}, \sigma_R, \alpha_R) .$$



↑*Quantics Tensor Train (QTT)*

- Algorithm to learn a function's QTT format:

Tensor Cross Interpolation (TCI) *[2].*

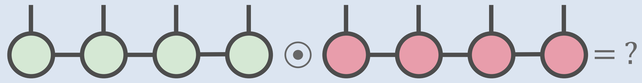
Benefits and Challenges of Using QTT

- *Why is TT good?* Ans: Storage reduction: $\mathcal{O}(n^d) \to \mathcal{O}(nd\chi^2)$; Fast contraction between TT-cores.
- *Why is QTT good?* Ans: Fast computation of integrals and derivatives of functions.

Challenges - TT Multiplication

Function multiplication arises in nonlinear PDEs, but computing it in the QTT format is challenging.

- Conventional algorithms - computational cost $\mathcal{O}(\chi^4)$.
- **Our goal: An efficient QTT multiplication algorithm computing the QTT of $g = f_1 \cdot f_2$.**



Quantics Tensor-Train Multiplication Algorithm (QTTM)

- **Our method: An iterative partial integration and reconstruction approach based on Tensor Cross Interpolation.**

Strategy: Iteratively compute TT-cores of $f_1 \cdot f_2$. At the d-th iteration:

Step 1:

- Given QTTs of f_1 and f_2 (in TCI form).
- Integrate the last $R - (d+n)$ quantics digits of f_1 and f_2 by contraction.

Step 2:

- Contract the cores (from d to $d+n$). The parameter n - chosen such that 2^n exceeds the maximum TT-rank desired.

Step 3:

- Elementwise multiplication between the derived TT of f_1 and f_2.

Step 4:

- Cross decomposition of the resulting tensor via rank-revealing LU.
- Form row / column interpolation sets for $f_1 f_2$ at dimension d and get the TT-core from the column-basis interpolation.

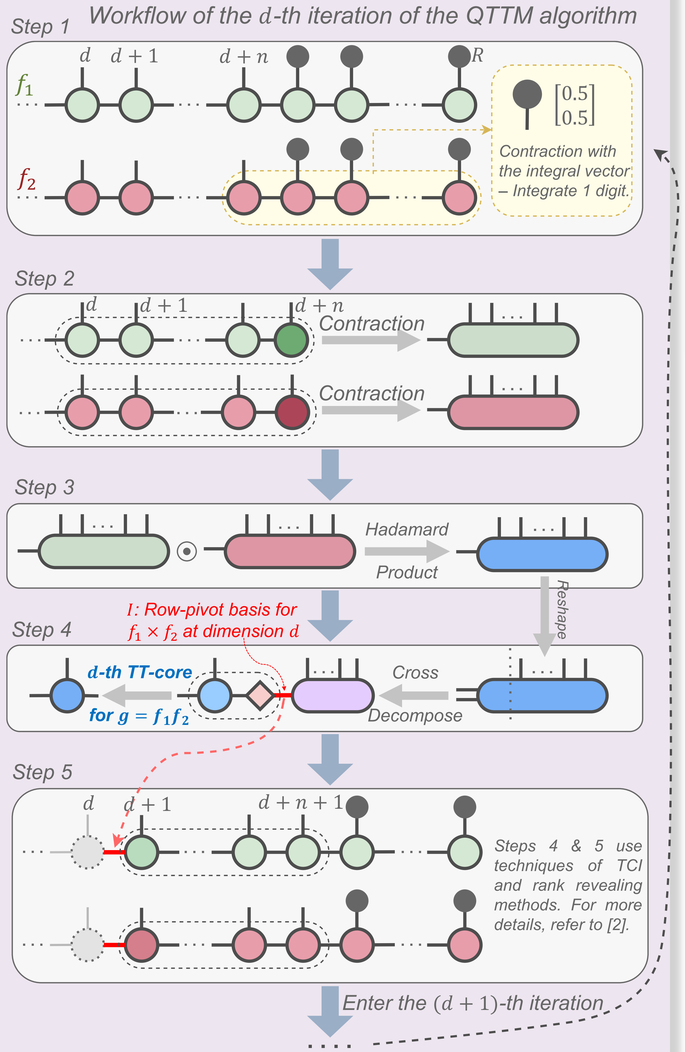
Step 5:

- The row interpolation set (I) - used to assemble the $(d+1)$-th TT-cores of f_1 and f_2, unifying the basis of the remaining TT of f_1 and f_2.

Why does this work?

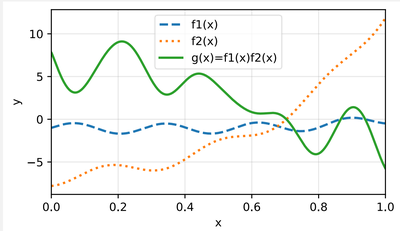
Partially reconstructing f_1 and f_2 on a coarser grid (via integration and contraction), reducing the computational complexity to $\mathcal{O}(\chi^3)$.

Workflow of the d-th iteration of the QTTM algorithm

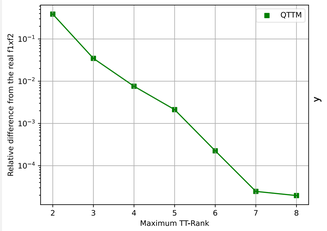


Numerical Results

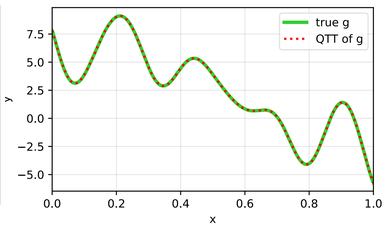
We test QTTM using two continuous functions f_1 and f_2 shown in the right figure. We first get QTTs of f_1 and f_2 via TCI as the inputs of QTTM.



QTTM returns the QTT of $g = f_1 \cdot f_2$ with TT-cores connected according to a specified maximum TT-rank (provided as an input argument).



- *Approximation error of the QTT as a function of TT-rank. High TT-ranks → accurate QTT of g.*



- *Comparison between the QTT approximation of g obtained from QTTM (TT-rank= 8) and the true g.*

Conclusion

- On synthetic function tests, the current QTTM algorithm produces accurate QTT of $f_1 \cdot f_2$. Higher TT-ranks lead to better approximations.
- Future work:
  1. Application in PDEs of fluid dynamics / quantum physics.
  2. More tricky cases - When will QTTM fail?
  3. Computational complexity / performance analysis.

References

[1] Takahashi, H., Sakurai, R., & Shinaoka, H. (2025). Compactness of quantics tensor train representations of local imaginary-time propagators. SciPost Physics, 18(1).

[2] Fernández, Y. N., Ritter, M. K., Jeannin, M., Li, J., Kloss, T., Louvet, T., Terasaki, S., Parcollet, O., Von Delft, J., Shinaoka, H., & Waintal, X. (2025). Learning tensor networks with tensor cross interpolation: New algorithms and libraries. SciPost Physics, 18(3).